Day 169: Bayesian Optimisation from Scratch

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 2 | Complete study guide

Watch: Gaussian Processes and Bayesian Optimization from Physicists' Perspective (Machine Learning in the Nanoworld) | Bayesian Optimization: expected improvement (+LCB, PI, EV) interpretation (A. Sala)

Script: `day169_bayesian_optimisation.py` · every number in this guide is copied by program from `day169_run_output.txt`, the real output of that script.

Grid and random search choose every trial blind. Bayesian optimisation learns from the trials so far: a Gaussian process predicts the score everywhere with an uncertainty, and expected improvement picks the next trial. On Day 168's 200 real boosting configurations it reached within 0.001 of the best in a median of 20 trials against random search's 44. With the scores shuffled, so there is nothing to learn, it fell back to random search's level (0.09 against 0.13 finding the best by 27 trials), which shows the gain comes from the structure the GP learns. Two honest complications: on a smooth real problem a purely greedy rule did as well, and on a 1-D test the GP was confidently wrong for five trials.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

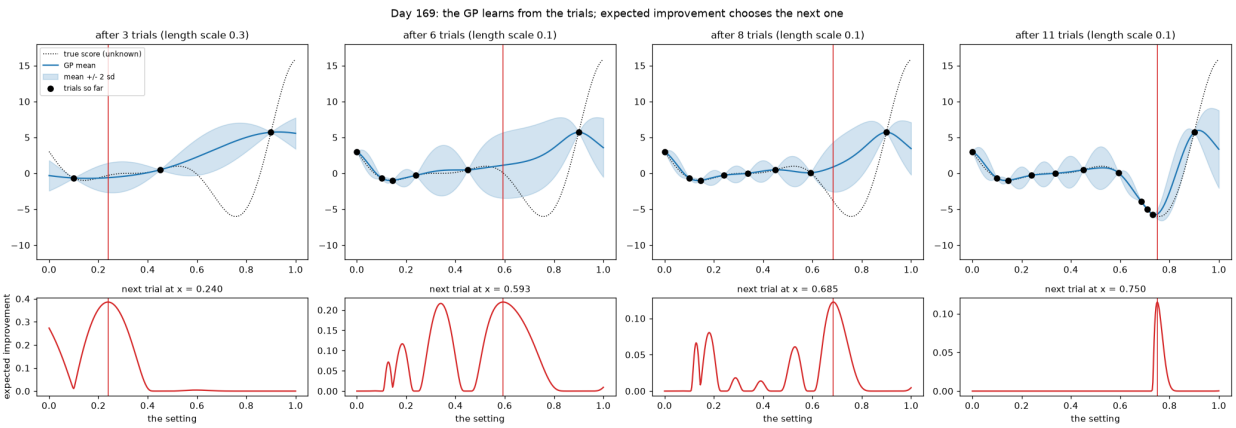
Bayesian optimisation tunes in a loop: fit a **surrogate model** to the trials so far, use it to choose the next trial, run it, repeat. The surrogate here is a **Gaussian process** (GP): for any setting it gives a predicted score (the mean) and how unsure it is (the standard deviation).

An **acquisition function** turns the mean and uncertainty into a choice. **Expected improvement** (EI) asks: if the score here is as uncertain as the GP says, how much better than the best so far should we expect it to be? A low predicted score helps; so does high uncertainty, because it might be much lower.

REAL-WORLD SCENARIO

An oil company drills test wells. After each one it updates a map of where oil probably is and how sure it is, then drills where the chance of a better find is highest: sometimes next to a good well, sometimes in an unexplored area. Each well is expensive, so thinking between wells pays.

SYSTEM DIAGRAM



Bayesian optimisation on a 1-D test function (dotted, unknown to the search). Top: the GP's mean and a band of 2 standard deviations; bottom: expected improvement, and its peak (red), the next trial. With 3 trials the GP misses the deep dip near 0.76; by 11 trials it has found it.

Today adds the second tool of Week 25's toolkit. It reuses Day 168 entirely: the same rows of UCI Adult (6,000 train each configuration, 6,000 others score it, 6,032 validation rows kept fresh), Day 109's boosting with 100 trees, the same search space and Day 168's parts, imported from `day168_exercise.py`. Everything else is written from scratch in NumPy; even the normal distribution's CDF is built from `math.erf`.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. A Gaussian process from scratch

DEFINITION — start here, no prior knowledge assumed

A GP assumes that settings close together have similar scores. The **kernel** says how similar: 1 for the same point, falling towards 0 with distance. The **length scale** sets how fast: a short one lets the score wiggle, a long one forces it to be smooth.

Given the trials, the GP's prediction at a new point is a weighted blend of the trial scores, weighted by similarity; its uncertainty is small near trials and large far from them. The length scale is chosen from 8 candidates by the **log marginal likelihood**: how probable the observed scores are under each, which rewards fitting the data and penalises needless wiggle.

REAL-WORLD SCENARIO

A weather map built from a few stations: near a station the temperature is known; between stations it is interpolated, and far from all stations it is a guess.

MATHEMATICS

$$k(x, x') = (1 + \sqrt{5}\, r + 5r^2/3)\, e^{-\sqrt{5}\, r}, \quad r = |x - x'| / l,\ l = \text{length scale} \quad \text{(Matérn 5/2)}$$

$$\text{mean}(x) = k_x^{\,T} K^{-1} z \quad \text{var}(x) = 1 - k_x^{\,T} K^{-1} k_x \quad (z = \text{standardised scores},\ K = \text{trials' kernel + noise})$$

$$\log p(z) = -\tfrac{1}{2}\, z^T K^{-1} z - \tfrac{1}{2} \log|K| - (n/2) \log 2\pi$$

IMPLEMENTATION

```
def matern52(A, B, ell):
    d = np.sqrt(np.maximum(((A[:, None, :] - B[None, :, :]) ** 2).sum(-1), 0)) / ell
    return (1 + np.sqrt(5) * d + 5 / 3 * d ** 2) * np.exp(-np.sqrt(5) * d)
def gp_fit(X, y, ell, noise=NOISE):
    m = y.mean(); sd = y.std() if y.std() > 0 else 1.0
    z = (y - m) / sd
    K = matern52(X, X, ell) + noise * np.eye(len(X))
    L = np.linalg.cholesky(K)
    a = np.linalg.solve(L.T, np.linalg.solve(L, z))
    lml = -0.5 * z @ a - np.log(np.diag(L)).sum() - 0.5 * len(X) * np.log(2 * np.pi)
```

```
    return dict(X=X, L=L, a=a, m=m, sd=sd, ell=ell, lml=lml)
def gp_predict(g, Xs):
    Ks = matern52(Xs, g["X"], g["ell"])
    mu = Ks @ g["a"]
    v = np.linalg.solve(g["L"], Ks.T)
    var = np.maximum(1 - (v * v).sum(0), 1e-12)
    return g["m"] + g["sd"] * mu, g["sd"] * np.sqrt(var)
def fit_gp(X, y, ells=ELLS):
    return max((gp_fit(X, y, e) for e in ells), key=lambda g: g["lml"])
```

ACTUAL OUTPUT

```
  two trials (0 -> 0, 1 -> 1), length scale 1: mean [0.0001, 0.5, 0.9999], sd [0.005, 0.1573, 0.005] (by hand: 0, 0.5, 1; sd
      at 0.5 = 0.5 x sqrt(1 - 2 x 0.8287^2 / 1.5240) = 0.157)
  Forrester, 4 trials: length scale 0.05: log marginal likelihood   -5.673
  Forrester, 4 trials: length scale 0.10: log marginal likelihood   -5.607
  Forrester, 4 trials: length scale 0.20: log marginal likelihood   -5.481
  Forrester, 4 trials: length scale 0.30: log marginal likelihood   -5.625
  Forrester, 4 trials: length scale 0.50: log marginal likelihood   -7.309
  Forrester, 4 trials: length scale 0.80: log marginal likelihood  -16.276
  Forrester, 4 trials: length scale 1.20: log marginal likelihood  -51.409
  Forrester, 4 trials: length scale 2.00: log marginal likelihood -265.629
  chosen length scale 0.2; mean at the trials [3.027, 0.115, -0.149, 15.829] against [3.027, 0.115, -0.149, 15.83] (it
      passes through them)
  largest uncertainty 4.624 at x = 0.198; true minimum -6.021 at x = 0.758
  Bayesian optimisation from trials at 0.1, 0.45 and 0.9; trials 4 to 15 at 0.240, 0.000, 0.145, 0.593, 0.338, 0.685, 0.710,
      0.733, 0.750, 0.765, 0.758, 0.758
  best score after each of trials 4 to 15: -0.66, -0.66, -0.99, -0.99, -0.99, -3.92, -5.01, -5.72, -5.99, -5.99, -6.02,
      -6.02
Actual output from this lesson's run.
```

What it means

By hand. With trials 0 → 0 and 1 → 1 the GP predicts 0.5 halfway, by symmetry, with standard deviation 0.1573, matching the hand calculation. At the trials themselves it gives 0.0001 and 0.9999 rather than exactly 0 and 1, with sd 0.005: the small noise term added for numerical safety.

Choosing the length scale. On 4 trials of the test function the marginal likelihood peaked at 0.2 (-5.481) and fell steeply for smoother GPs (-265.629 at 2.0). The chosen GP passes through every trial; its largest uncertainty, 4.624, is in the widest gap.

A first-draft finding: the GP can be confidently wrong. Starting from trials at 0.1, 0.45 and 0.9, Bayesian optimisation spent trials 4 to 8 away from the dip (0.240, 0.000, 0.145, 0.593, 0.338) because the GP drew a smooth curve between 0.45 and 0.9 and missed the dip near 0.76 (diagram, first panel). Trial 7, at 0.593, narrowed that gap; with it, the GP's uncertainty beyond 0.6 still allowed much lower scores (third panel), and trial 9 went to 0.685, inside the dip. From there the trials walked down: best -3.92, -5.01, -5.72, then -6.02, the true minimum (-6.021). Trials 14 and 15 repeated the same point; a likely reason, not tested here, is that the small noise term leaves a little expected improvement at the best point.

WHY CHOOSE THIS?

A GP gives an uncertainty with every prediction, and with few trials it is cheap and exact.

WHY NOT THIS? (TRADEOFFS)

It assumes smoothness everywhere; a sharp dip between trials can be invisible to it. Its cost grows as n^3.

2. Expected improvement: the formula against simulation

DEFINITION — start here, no prior knowledge assumed

If the GP's belief about the score at a point is normal with mean μ and standard deviation σ, the improvement over the best score so far is max(best – score, 0). Its average has a closed form. The check: draw a million scores from the belief and average the improvement directly.

REAL-WORLD SCENARIO

Choosing which lottery to enter: one has a reliable small prize, another an uncertain larger one. Expected winnings, not just the most likely outcome, decide.

MATHEMATICS

$$EI = (best - \mu)\,\Phi(z) + \sigma\,\phi(z), \quad z = (best - \mu) / \sigma$$

$$\Phi = \text{normal CDF} = \tfrac{1}{2}(1 + erf(z/\sqrt{2})) \quad \phi = \text{normal density}$$

IMPLEMENTATION

```
def expected_improvement(mu, sd, best):
    z = (best - mu) / sd
    return (best - mu) * normal_cdf(z) + sd * normal_pdf(z)
def propose(X, y, C):
    g = fit_gp(X, y)
    mu, sd = gp_predict(g, C)
    return int(np.argmax(expected_improvement(mu, sd, y.min()))), g
```

ACTUAL OUTPUT

```
  mean +0.0, sd 1.0, best 0.0: formula 0.39894, simulation 0.39821
  mean +1.0, sd 0.5, best 0.0: formula 0.00425, simulation 0.00419
  mean -0.5, sd 0.2, best 0.0: formula 0.50040, simulation 0.50058
  mean +0.3, sd 2.0, best 0.0: formula 0.65684, simulation 0.65507
  (the first: 0.39894 = 1 / sqrt(2 pi), by hand)
Actual output from this lesson's run.
```

What it means

Formula and simulation agree to the simulation's precision: 0.39894 against 0.39821, and 0.65684 against 0.65507. The pairs show EI's two sources of value: a point predicted 0.5 below the best with little doubt is worth 0.50040, while a point predicted 0.3 *above* the best but very uncertain (sd 2) is worth more, 0.65684. That is exploration, built into one number.

WHY CHOOSE THIS?

EI has a closed form, needs no tuning knob, and balances a good mean against a large uncertainty.

WHY NOT THIS? (TRADEOFFS)

It trusts the GP's uncertainty; when that is wrong (section 1), EI is confidently wrong too.

3. A known-truth benchmark: Hartmann-3

DEFINITION — start here, no prior knowledge assumed

Hartmann-3 is a standard test function on the unit cube with four dips and a known minimum, -3.86278; the script checks that value first. Three methods get 40 trials: random search (1,000 runs), Bayesian optimisation with EI, and a **greedy** version that always picks the lowest predicted mean, ignoring uncertainty (50 runs each). The first 5 trials of each Bayesian run are random.

The next trial is chosen among 2,000 random candidates plus 500 near the best trial so far.

REAL-WORLD SCENARIO

A hiker looking for the lowest valley in fog: always walking downhill finds a valley, but not necessarily the lowest; sometimes it pays to look over the next ridge.

ACTUAL OUTPUT

```
  check: the function at its known minimiser -3.86278
  10 trials: median gap from the minimum: EI 0.4333, greedy 0.8645, random 1.0322
  20 trials: median gap from the minimum: EI 0.0906, greedy 0.1293, random 0.6604
  30 trials: median gap from the minimum: EI 0.0475, greedy 0.1178, random 0.5043
  40 trials: median gap from the minimum: EI 0.0111, greedy 0.0856, random 0.3968
  ei    : runs still more than 0.1 from the minimum after 40 trials: 0.12
  mean  : runs still more than 0.1 from the minimum after 40 trials: 0.48
  random: runs still more than 0.1 from the minimum after 40 trials: 0.92
  (12 s)
Actual output from this lesson's run.
```

What it means

After 40 trials the median EI run was 0.0111 from the minimum, greedy 0.0856 and random 0.3968. Greed gets stuck: 0.48 of greedy runs were still more than 0.1 away after 40 trials, against 0.12 for EI and 0.92 for random search. Ignoring uncertainty means never checking whether another dip is deeper.

WHY CHOOSE THIS?

A benchmark with several dips shows whether exploration is needed, with the right answer known.

WHY NOT THIS? (TRADEOFFS)

Test functions are cheap and smooth; real tuning problems may have noise and flat regions they lack.

4. Day 168's 200 real configurations

DEFINITION — start here, no prior knowledge assumed

To compare fairly with Day 168, Bayesian optimisation chooses among the same 200 random configurations, recomputed here from the same seed. Each choice reveals a score that is already known, so 100 runs of each variant cost seconds. Random search is Day 168's 1,000 random orders of the pool.

Two controls. **Shuffled scores:** the 200 scores are shuffled among the configurations, so there is no pattern for the GP to learn; if Bayesian optimisation still won, its advantage would be an artefact. **What the GP knows:** fitted on 30 random configurations, how well does it predict the other 170?

REAL-WORLD SCENARIO

A wine taster who has tried 30 wines from a region can guess which of the other 170 bottles are best, if the region's wines follow a pattern. With labels shuffled, tasting teaches nothing.

ACTUAL OUTPUT

```
  Day 168's pool recomputed in 347 s: best 0.2918, worst 0.7371 (Day 168 printed 0.2918 and 0.7371)
  (400 Bayesian optimisation runs of 64 trials in 26 s)
  random search (1000)          : median best at 8 / 16 / 27 / 64 trials 0.2977 / 0.2958 / 0.2949 / 0.2924; found the pool's
      best by 27: 0.13, by 64: 0.33
  BO, expected improvement      : median best at 8 / 16 / 27 / 64 trials 0.2962 / 0.2945 / 0.2924 / 0.2918; found the pool's
      best by 27: 0.43, by 64: 0.99
  BO, greedy (lowest mean)      : median best at 8 / 16 / 27 / 64 trials 0.2961 / 0.2945 / 0.2918 / 0.2918; found the pool's
      best by 27: 0.61, by 64: 0.95
  BO, explore only (largest sd) : median best at 8 / 16 / 27 / 64 trials 0.2999 / 0.2961 / 0.2924 / 0.2918; found the pool's
      best by 27: 0.40, by 64: 0.66
  BO on shuffled scores         : median best at 8 / 16 / 27 / 64 trials 0.2970 / 0.2957 / 0.2949 / 0.2924; found the pool's
      best by 27: 0.09, by 64: 0.33
  trials until within 0.001 of the pool's best: EI median 20, random median 44
```

ACTUAL OUTPUT

```
  the GP fitted on 30 configurations, predicting the other 170 (20 repeats): correlation 0.74, rank correlation 0.84, length
      scale chosen 0.3; its predicted best is in the true best 10% in 0.70 of repeats
Actual output from this lesson's run.
```

What it means

Reproduced first. The recomputed pool gave best 0.2918 and worst 0.7371, exactly Day 168's numbers: same seeds, same rows, same scores.

Bayesian optimisation wins on real data. With EI it found the pool's best configuration within 27 trials in 0.43 of runs and within 64 in 0.99; random search in 0.13 and 0.33. It needed a median 20 trials to get within 0.001 of the best, random search 44.

The control behaves. On shuffled scores Bayesian optimisation found the best by 27 trials in 0.09 of runs and by 64 in 0.33, random search's level. The GP's advantage comes from real structure: fitted on 30 configurations it predicts the other 170 with rank correlation 0.84, and its predicted best is in the true best 10% in 0.70 of repeats.

An honest complication: greed did well here. The greedy rule found the best by 27 trials more often than EI (0.61 against 0.43), though less often by 64 (0.95 against 0.99). Exploring only (largest uncertainty) was worst of the GP rules (0.66 by 64). A likely reason, not tested here, is that this pool has one broad good region (Day 168's scatter), so going straight downhill works; on Hartmann-3, with several dips, greed got stuck.

WHY CHOOSE THIS?

Pool-based runs make 100 repeats nearly free, and the shuffled control rules out luck.

WHY NOT THIS? (TRADEOFFS)

Choosing from 200 fixed points is easier than searching a continuous space; section 5 drops that limit.

5. Proposing new configurations, fitted for real

DEFINITION — start here, no prior knowledge assumed

Now Bayesian optimisation may propose any point in the cube, and each proposal is fitted for real: 3 runs of 30 trials, each starting from 5 random trials. Depth and min leaf are whole numbers, so two nearby points can decode to the same configuration; such a repeat would reuse the earlier result but still spend a trial.

Each run's winner is also scored once on the 6,032 fresh rows.

REAL-WORLD SCENARIO

A chef who has tasted 30 versions of a sauce may invent a 31st that is better than any of them; whether it is really better is checked by people who did not taste the first 30.

ACTUAL OUTPUT

```
  run 1: best of its 5 random starting trials 0.2962; best tuning log loss 0.2924 at trial 20 (learning rate 0.118, depth 4,
      min leaf 15); fresh rows 0.2968; repeated configurations 0; GP time 0.1 s against fitting 52 s
  run 2: best of its 5 random starting trials 0.2916; best tuning log loss 0.2910 at trial 26 (learning rate 0.128, depth 4,
      min leaf 1); fresh rows 0.2995; repeated configurations 0; GP time 0.1 s against fitting 50 s
  run 3: best of its 5 random starting trials 0.3008; best tuning log loss 0.2915 at trial 22 (learning rate 0.138, depth 4,
      min leaf 1); fresh rows 0.2999; repeated configurations 0; GP time 0.1 s against fitting 49 s
  random search of 30 trials (from Day 168's pool): median 0.2949; 10% of searches reach 0.2918; the pool's best 0.2918, its
      fresh score 0.3010
Actual output from this lesson's run.
```

What it means

All three runs ended below random search's median for 30 trials (0.2949): at 0.2924, 0.2910 and 0.2915. Two found configurations better than any in Day 168's pool (best 0.2918), all near learning rate 0.12–0.14 and depth 4. No proposal repeated a configuration.

How much was the search, and how much the start? The added line answers it. Run 2's five random starting trials already reached 0.2916, so the search improved it by little. Run 3 started at 0.3008 and the

search brought it to 0.2915; run 1 from 0.2962 to 0.2924. Reporting only the final scores would have credited the search with run 2's luck.

Fresh rows. The three winners scored 0.2968, 0.2995 and 0.2999 on the fresh rows; the pool's best scored 0.3010. Their order on fresh rows is not their order on tuning rows (the run 2 winner was best on tuning rows, run 1's on fresh rows): differences of a few thousandths are within the noise of 6,000 scoring rows, Day 168's winner's curse again.

Cost. The GP took about 0.1 s per run against about 50 s of model fitting (52 s, 50 s, 49 s). A first run of this script reported 29,907 s of fitting for run 3: the computer went to sleep during it, and the clock counted the sleep. The scores were identical; the script was rerun with the computer kept awake, and that run is the one shown.

WHY CHOOSE THIS?

Proposing anywhere can find settings better than any in a fixed pool, at a GP cost far below one model fit.

WHY NOT THIS? (TRADEOFFS)

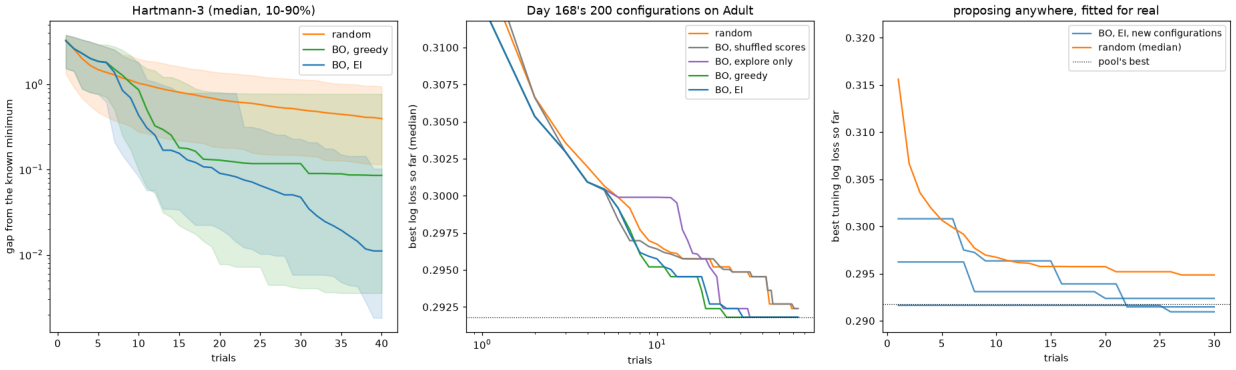
Each run is one sample; three runs show the range, not a precise average. Whole-number settings make the GP's smooth picture an approximation.

Today's part for the week's build

`day169_exercise.py` holds the GP and its acquisition: `normal_cdf`, `normal_pdf`, `matern52`, `gp_fit`, `gp_predict`, `fit_gp`, `expected_improvement` and `propose`, copied unchanged. Its hand checks:

ACTUAL OUTPUT

```
normal_cdf 0, 1.96: [0.5, 0.975] (by hand 0.5, 0.975); normal_pdf(0) 0.39894 (1 / sqrt(2 pi) =
    0.39894)
matern52 at distances 0, 0.5, 1 (length scale 1): [1.0, 0.8286, 0.524] (by hand 1, (1 + 1.1180 +
    0.4167) e^-1.1180 = 0.82865, (1 + 2.2361 + 1.6667) e^-2.2361 = 0.5240)
GP on trials 0 -> 0 and 1 -> 1: mean [0.0, 0.5, 1.0] (by hand 0, 0.5, 1), sd at 0.5 0.157 (0.5 x
    sqrt(1 - 2 x 0.8287^2 / 1.5240) = 0.157)
fit_gp on 8 points of sin(3x): length scale 0.3, its log marginal likelihood equals the largest of
    all 8: True
expected_improvement(mean 0, sd 1, best 0) 0.39894 (by hand 0.39894); (mean -1, sd 0.001, best 0)
    1.0000 (by hand 1: a sure improvement of 1)
propose with trials 0 -> 1 and 1 -> 0, candidates 0.05 and 0.95: index 1 (by hand 1: next to the
    better trial)
```



Left: Hartmann-3, gap from the known minimum (median and 10–90%). Middle: Day 168's pool, median best so far for each method. Right: three Bayesian runs proposing new configurations, against random search's median.

Interview preparation — technical questions

Q: How would you show that Bayesian optimisation's gain is real?

A: A control with the structure removed: shuffling the scores among configurations dropped it to random search's level (0.09 against 0.13).

Q: Why might a greedy rule beat expected improvement?

A: On a problem with one broad good region, going downhill is enough; it fails on problems with several dips (0.48 of greedy runs stuck on Hartmann-3).

Q: How do you choose the GP's length scale?

A: Maximise the log marginal likelihood over candidates; it trades fit against complexity.

Q: How expensive is the GP compared with the model?

A: Here a fraction of a second per run against tens of seconds of fitting; the GP's n^3 cost matters only with thousands of trials.

Q: Your GP never samples a region. What could be wrong?

A: It may be confidently wrong: smooth interpolation between distant trials hides a dip. More random start points or a shorter length scale help.

Interview preparation — theoretical questions

Q: What does a Gaussian process give that a regression model does not?

A: A full predictive distribution: a mean and an uncertainty at every point, which acquisition functions need.

Q: What is the exploration-exploitation trade-off in EI?

A: EI grows when the mean is low (exploitation) and when the standard deviation is high (exploration).

Q: When is Bayesian optimisation worth it?

A: When each trial is expensive and there are few settings; with cheap trials or many dimensions, random search is simpler.

Q: Why standardise the scores before fitting the GP?

A: The kernel's prior has mean 0 and variance 1; standardising makes that prior sensible for any score scale.

Key takeaway and what's next

> **KEY TAKEAWAY**
> Bayesian optimisation spends a little thought between trials to choose them better. On real boosting it reached the best configuration about twice as fast as random search, and a shuffled-score control showed the gain is real. It can be confidently wrong when trials are few, and on a smooth problem plain greed can match it.

- Pool: within 0.001 of the best in 20 trials (median) against random search's 44.
- Shuffled control: 0.09 against random search's 0.13.
- Hartmann-3 after 40 trials: EI 0.0111, greedy 0.0856, random 0.3968 from the minimum.
- Expected improvement's formula matched simulation: 0.39894 against 0.39821.

Suggested watch order, if starting from zero

- 1. Gaussian Processes and Bayesian Optimization from Physicists' Perspective (Machine Learning in the Nanoworld) — Gaussian processes and Bayesian optimisation, introduced.
- 2. Bayesian Optimization: expected improvement (+LCB, PI, EV) interpretation (A. Sala) — what expected improvement means, beside other acquisition functions.

Day 170 preview

- **Learning-rate finders.** Raise the learning rate step by step during one short run, read a good value off the loss curve, and check that choice against full training runs.